POSTECH · GRUPO 119 · FASE 4

ToggleMaster

Relatório de entrega · Observabilidade, APM e Self-Healing

Integrantes

Nome completo	Matrícula	Usuário
Julio Cesar Diniz Nogueira	RM373719	JulioDinizN

1. Acesso aos entregáveis

- Repositório principal — código, instrumentação, CI e infraestrutura
- Repositório GitOps — estado desejado do Kubernetes e stack de observabilidade
- Documentação técnica

Vídeo de demonstração (20min05s): https://drive.google.com/drive/folders/15-CMmU3onMTld9xLVTTpMwazbzAwOhKI?usp=sharing (arquivo `togglemaster-fase4-demo.mp4`).

Tempo	Parte do vídeo	Requisito demonstrado
00:00	Apresentação	Contexto e objetivo da fase
01:18	Visão geral	Cluster OKE criado por Terraform (03:10 e 03:50), Argo CD com as 12 aplicações e a nova stack (04:15), pipeline de CI (04:52)
05:20	Arquitetura OpenTelemetry	Roteamento do Collector no diagrama e na configuração (07:22)
07:29	Grafana e Loki	Dashboard customizado (07:37) e busca de logs no Loki (08:57)
09:28	APM (Datadog)	Service Map com os cinco serviços (09:40), requisição de teste (10:37) e trace distribuído (11:08)
11:26	Incidente e Self-Healing	Falha injetada (13:10), erros no dashboard e no Loki (13:50), pod substituído (15:15), Discord (15:25), PagerDuty (15:50), log do Self-Healing (16:25), recuperação (16:05) e incidente resolvido (17:10)
17:22	Decisões e entregáveis	Justificativas e repositórios

2. Arquitetura de observabilidade

O cluster Kubernetes é o **OKE, da Oracle Cloud (OCI)**, usado no lugar do EKS citado no enunciado. A substituição de provedor foi aceita pela disciplina nas fases anteriores e vale para toda a base: Terraform, CI/CD, GitOps com Argo CD e os serviços gerenciados.

Os cinco microsserviços usam as bibliotecas do OpenTelemetry: SDK em Go no `auth-service` e no `evaluation-service`, instrumentação automática em Python (`opentelemetry-instrument` no Dockerfile) no `flag-service`, no `targeting-service` e no `analytics-service`. Nenhum serviço conhece Prometheus, Loki ou o APM: todos enviam métricas, logs e traces por OTLP/HTTP para o **OpenTelemetry Collector**, que roda como DaemonSet no namespace `observability`.

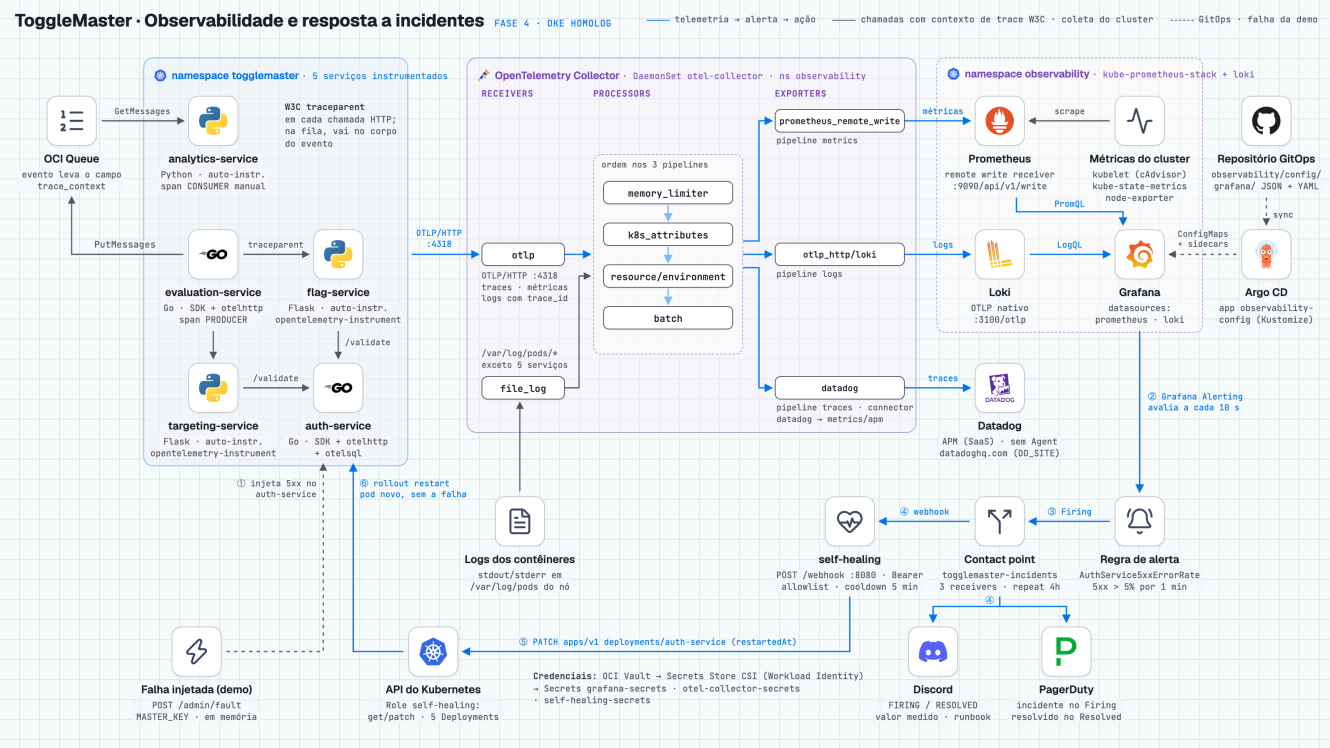
O Collector é a peça central. Recebe OTLP dos serviços e lê os logs de todos os demais contêineres do cluster (`file_log`), enriquece tudo com metadados do Kubernetes (`k8s_attributes`) e roteia por três pipelines:

Sinal	Exporter do Collector	Destino
Métricas	`prometheus_remote_write`	Prometheus, que também coleta kubelet, kube-state-metrics e node-exporter
Logs	`otlp_http/loki`	Loki (ingestão OTLP nativa)
Traces	`datadog`, com o connector `datadog/connector` para as estatísticas de APM	Datadog (Distributed Tracing e Service Map)

O contexto de trace W3C (`traceparent`) acompanha as chamadas HTTP do `evaluation-service` para `flag-service` e `targeting-service`, e destes para o `auth-service`. Na fila, o `evaluation-service` grava o contexto no campo `trace_context` do evento e o `analytics-service` continua o mesmo trace ao consumir, o que coloca os cinco serviços no Service Map.

O Grafana lê Prometheus e Loki. Dashboard, regra de alerta e contact points são arquivos versionados no repositório GitOps, carregados por sidecars. A regra `AuthService5xxErrorRate` (5xx do `auth-service` acima de 5% por 1 minuto) aciona um contact point com três destinos: incidente no PagerDuty, mensagem no Discord e o webhook do Self-Healing, que executa o equivalente a `kubectl rollout restart deployment/auth-service` pela API do Kubernetes.

Toda a stack (kube-prometheus-stack, Loki e OpenTelemetry Collector) é instalada pelo Argo CD a partir de charts Helm com versão fixa no repositório GitOps. As credenciais externas ficam no OCI Vault e chegam aos pods por Secrets Store CSI e Workload Identity.



Fluxo de telemetria pelo OpenTelemetry Collector e ciclo de alerta, incidente, ChatOps e Self-Healing.

3. Justificativa técnica

OpenTelemetry Collector como peça central

Com o Collector no meio, os serviços dependem só do padrão OTLP. Trocar ou acrescentar um backend, inclusive o APM, muda um exporter na configuração do Collector, sem alterar código nem reconstruir imagens. O Collector também resolve em um único ponto a coleta dos logs de todos os contêineres e a adição de metadados do Kubernetes. Usamos o OTel Collector padrão; o Grafana Alloy era opcional.

APM: Datadog em vez de New Relic

Escolhemos o Datadog pelo APM: o Service Map é montado a partir dos spans e mostra as dependências entre os serviços, e o trace distribuído traz cada chamada HTTP e cada consulta SQL como um recurso com tempo próprio. O Collector envia os traces pelo exporter `datadog`, e o connector `datadog` calcula as estatísticas de APM (taxa, erros e latência por serviço e recurso) a partir dos mesmos spans. Não há Datadog Agent nos nós nem biblioteca proprietária nos serviços, que continuam só com OpenTelemetry.

O New Relic também atenderia e receberia os traces por um exporter OTLP genérico; o Datadog pede exporter e connector próprios. Aceitamos esse acoplamento porque ele fica isolado em um arquivo de configuração do Collector: trocar de APM continua sendo trocar um exporter, sem alterar código nem reconstruir imagens.

Gestão de incidentes: PagerDuty em vez de OpsGenie

O Grafana Alerting tem integração nativa com o PagerDuty pela Events API v2: o incidente é aberto quando o alerta entra em `Firing` e resolvido quando volta ao normal, sem código intermediário. A Atlassian anunciou o encerramento do OpsGenie e deixou de aceitar novos clientes, portanto não faz sentido adotá-lo em um projeto novo.

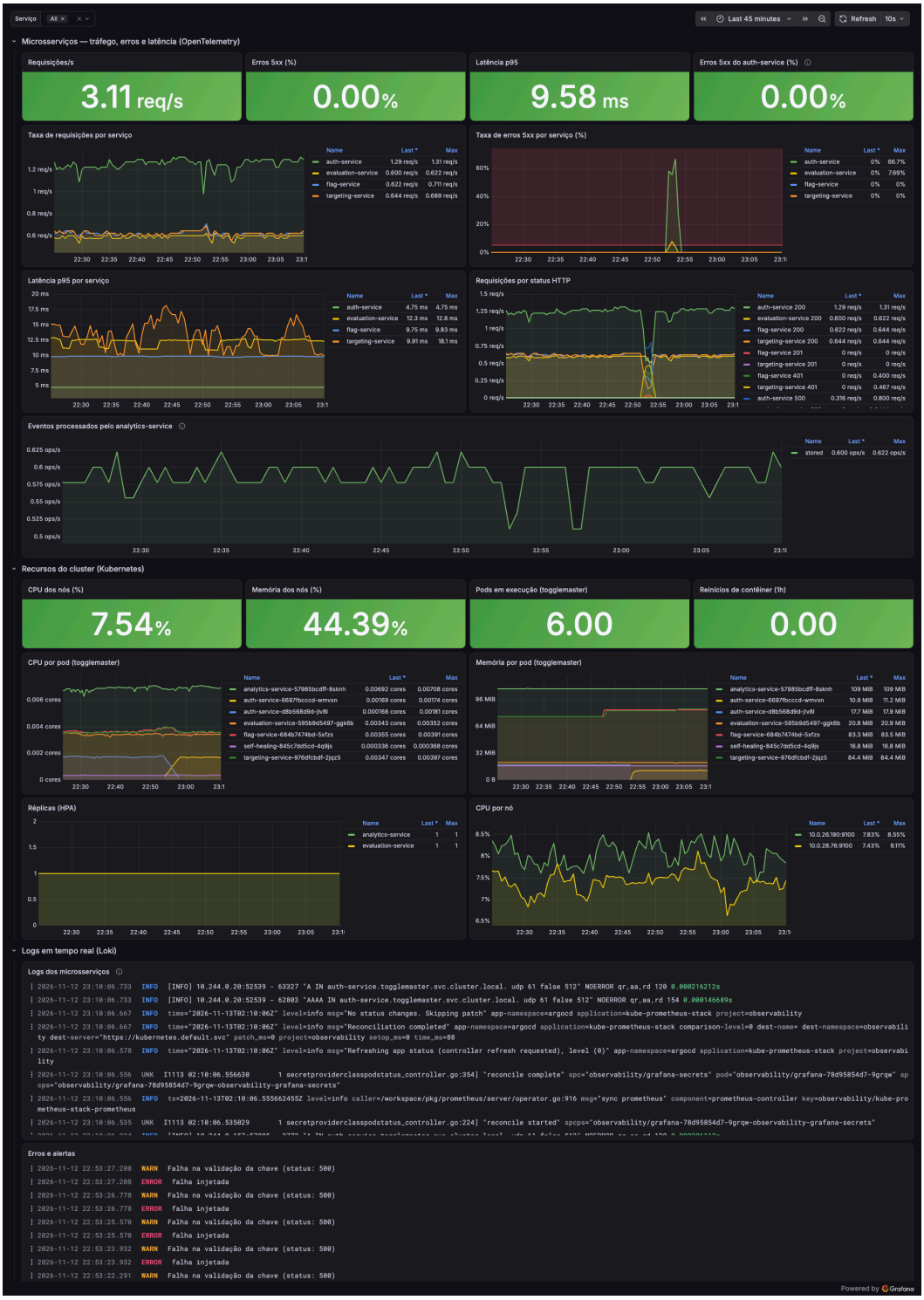
Self-Healing dentro do cluster

A automação é um receptor de webhook que roda no próprio cluster (`services/self-healing`) e é chamado pelo alerta. A API do OKE só aceita os CIDRs autorizados; uma função externa ou um runner hospedado do GitHub exigiriam abrir esse acesso de rede e distribuir credenciais do cluster fora dele. O receptor interno evita as duas coisas: usa uma ServiceAccount cujo Role permite apenas `get` e `patch` nos cinco Deployments dos serviços, valida um token do Vault, só age em Deployments de uma lista permitida e aplica um intervalo mínimo de 5 minutos entre reinícios (controle em memória do processo, suficiente para evitar laços de reinício).

4. Evidências

Capturas do ambiente de homologação (cluster OKE `togglemaster-oke`) feitas na noite de 12/11/2026, durante e logo após a gravação do vídeo. Os horários das telas estão em Brasília (UTC−3); os dos logs, em UTC, já no dia 13/11. O incidente foi provocado injetando falha no `auth-service` às 22:51; o alerta disparou às 22:53:10 (01:53:10 UTC) e foi resolvido às 22:54:10.

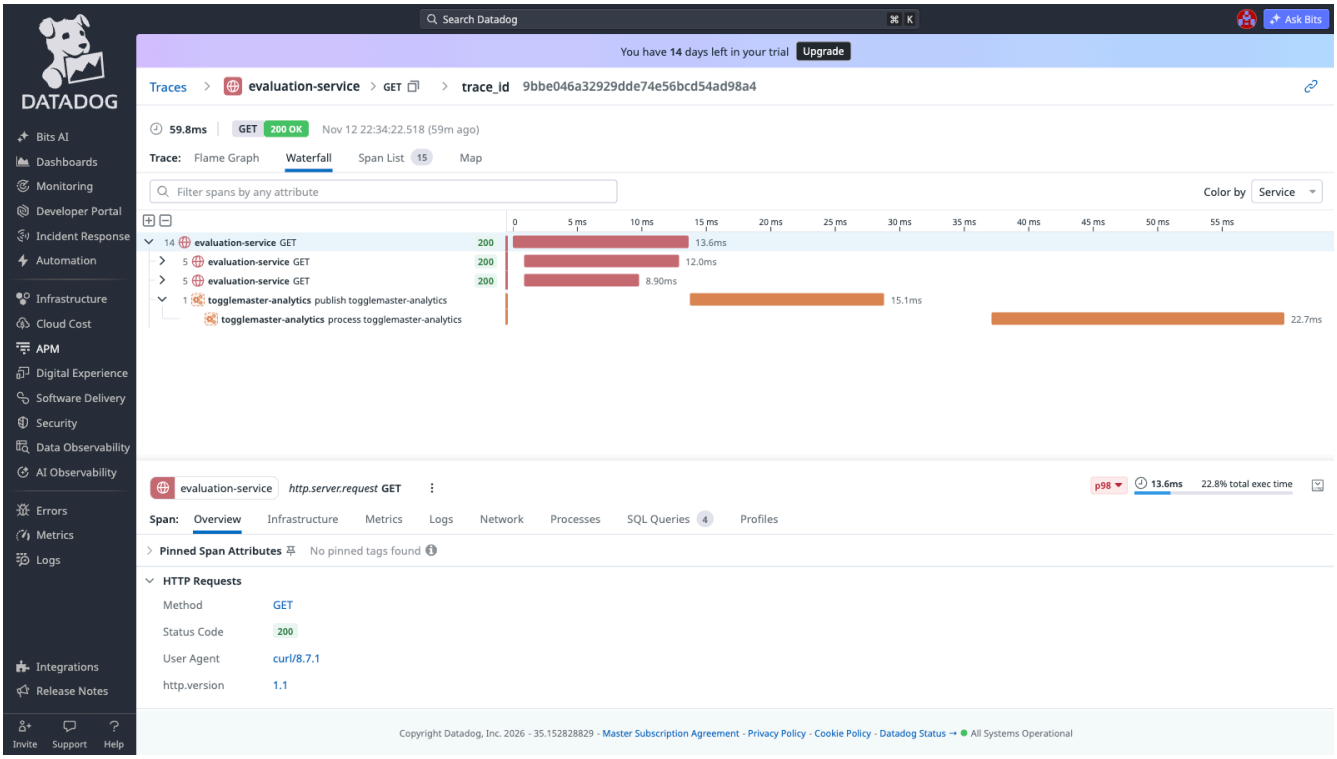
4.1 Dashboard customizado no Grafana



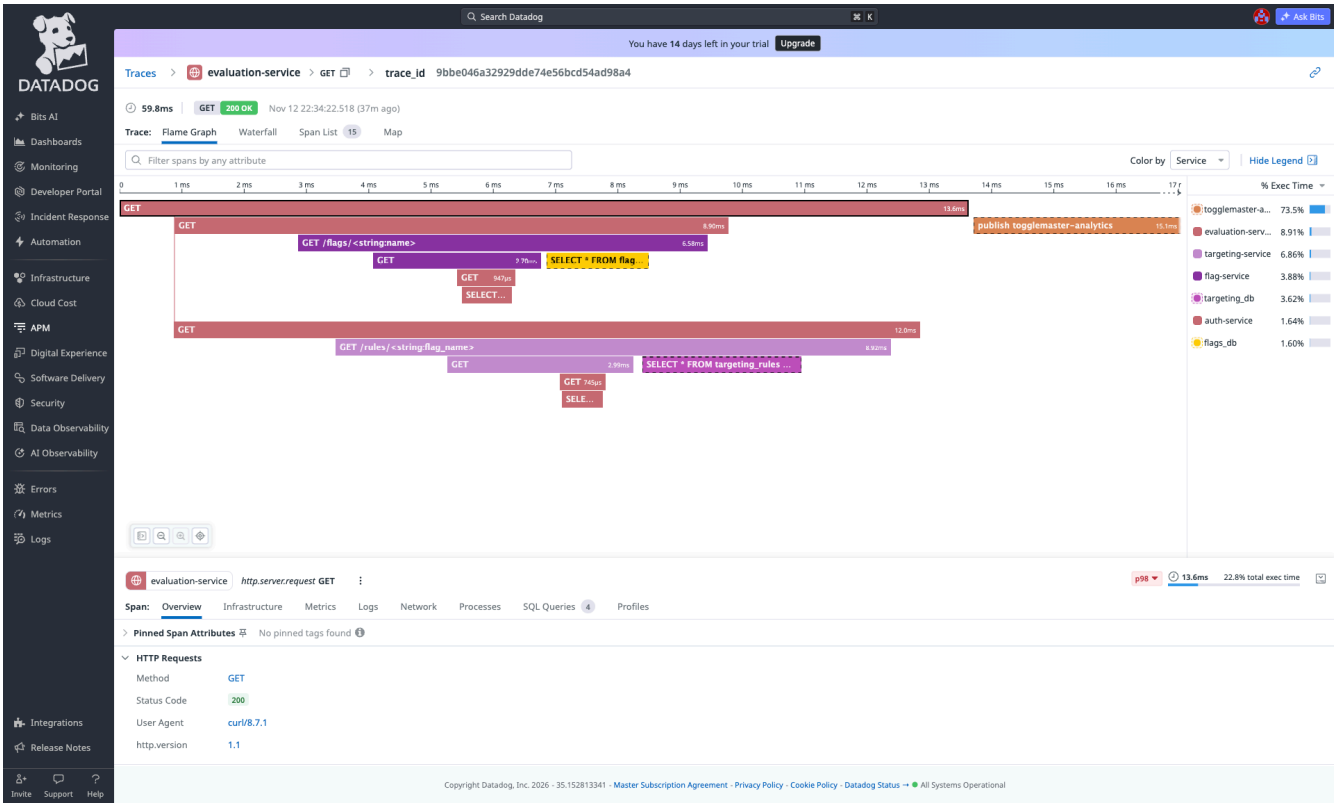
Dashboard “ToggleMaster — Saúde do Ecossistema”: tráfego, erros e latência por microsserviço, eventos processados pelo analytics-service, recursos do cluster e logs em tempo real. O pico de erros 5xx às 22:53 é o incidente da demonstração.

4.2 Trace distribuído no APM (Datadog)

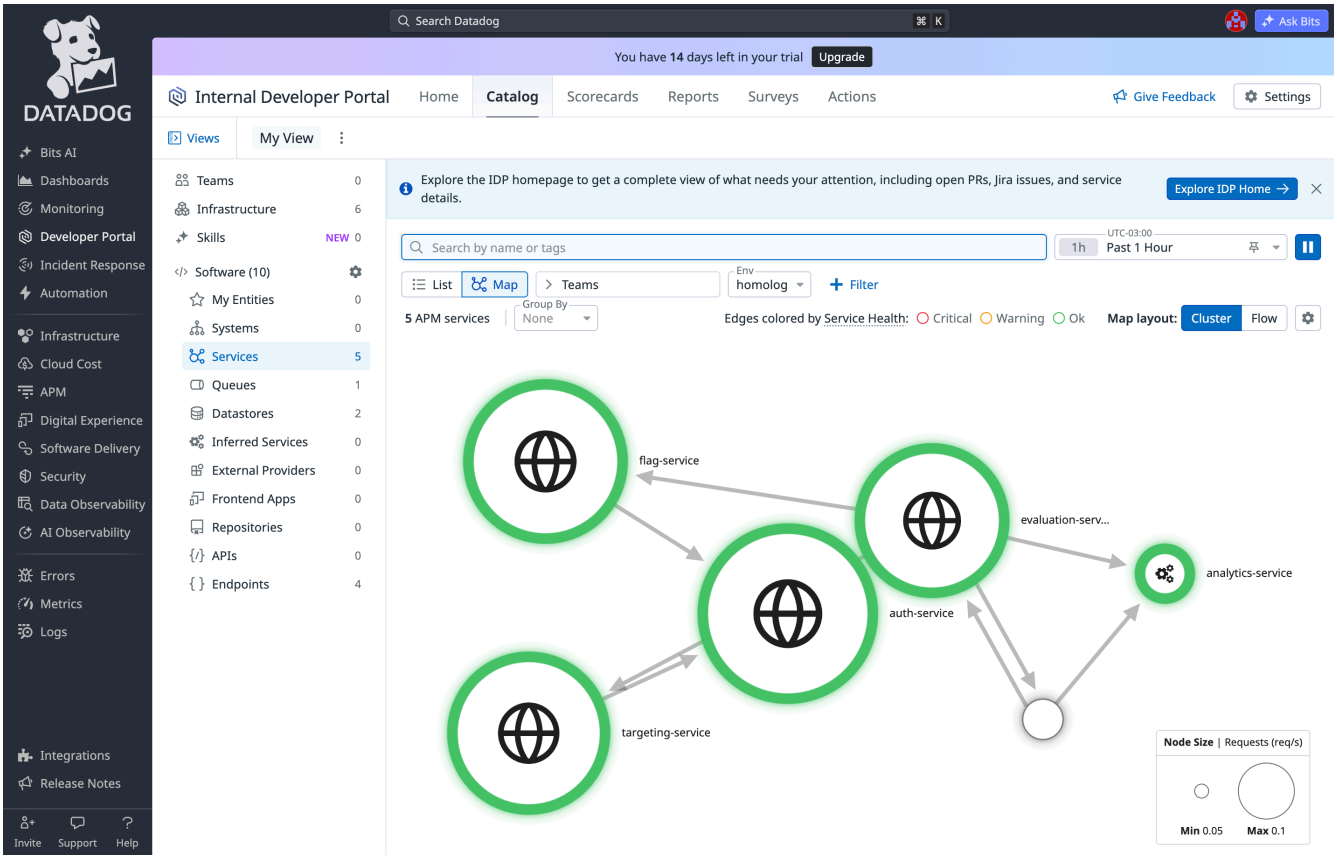
Trace da requisição de teste feita com `curl` ao `GET /evaluate` às 22:34:22 (`trace_id 9bbe046a32929dde74e56bcd54ad98a4`, 15 spans).



Visão em cascata do trace completo: a entrada no evaluation-service, as duas chamadas HTTP (flag-service e targeting-service, cada uma com 5 spans) e a passagem pela fila, com `publish` pelo evaluation-service e `process` pelo analytics-service. O painel inferior mostra o `User Agent curl` da requisição de teste.

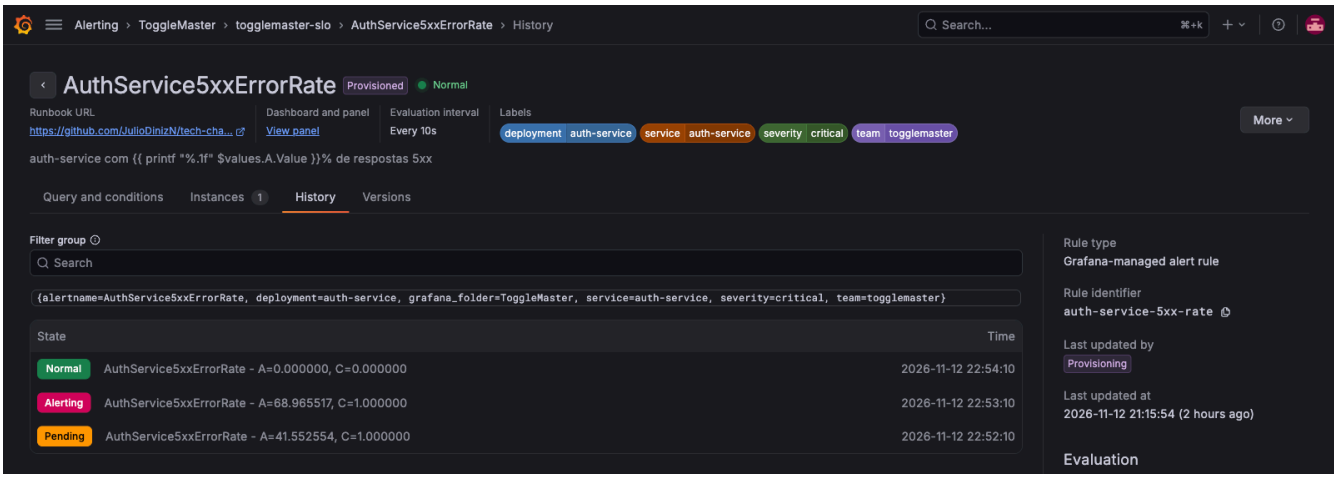


Detalhe dos primeiros 17 ms do mesmo trace em flame graph: evaluation-service, flag-service, targeting-service, auth-service e as consultas SQL aos bancos, com o tempo de cada etapa.



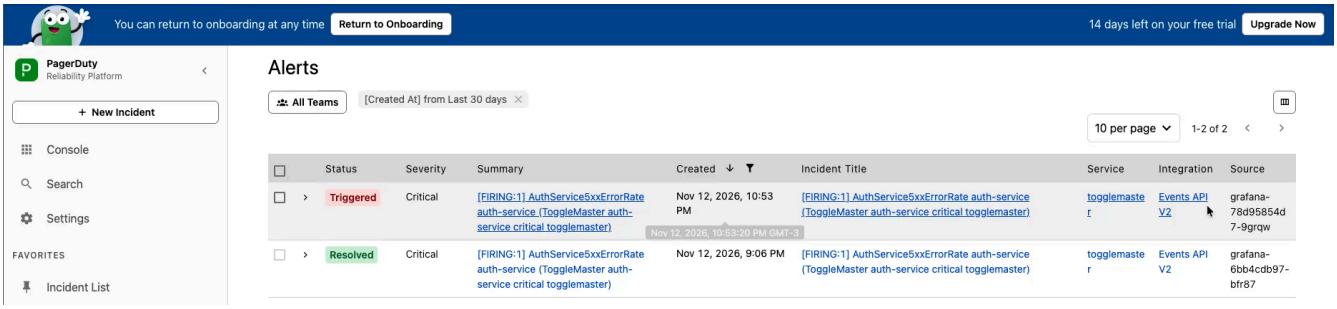
Service Map do ambiente `homolog` com os cinco microsserviços e a fila.

4.3 Alerta em Firing no Grafana

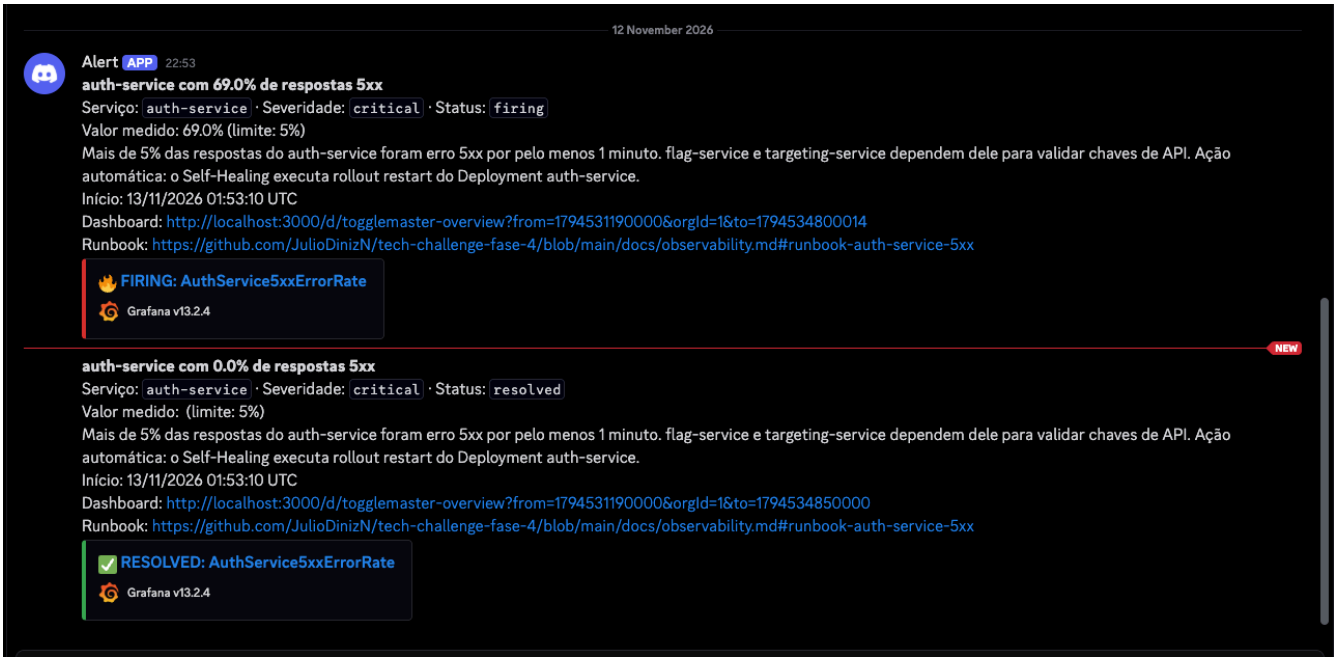


Histórico de estados da regra `AuthService5xxErrorRate` no Grafana Alerting para o incidente da gravação: `Pending` às 22:52:10 (41,6% de 5xx), `Alerting` (Firing) às 22:53:10 (69,0%) e `Normal` às 22:54:10, depois do Self-Healing. No vídeo, a página da regra não foi recarregada durante o incidente e por isso exibe o estado anterior; o disparo aparece ali pelo painel de erros, pelo Discord e pelo PagerDuty.

4.4 Incidente no PagerDuty e notificação no ChatOps (Discord)



PagerDuty durante a gravação (quadro do vídeo em 15:50): alerta `Triggered` criado às 22:53:20 pela integração Events API V2, com o incidente aberto automaticamente no serviço `togglemaster`. A linha `Resolved` é o ensaio feito mais cedo, às 21:06.



Mensagens enviadas pelo Grafana Alerting ao canal: disparo com 69% de respostas 5xx e resolução após o Self-Healing, com serviço, severidade, valor medido, horário, dashboard e runbook. O link do dashboard aponta para `localhost:3000` porque o Grafana da homologação não é exposto na internet e é acessado por `port-forward`.

4.5 Execução da automação de Self-Healing



Saída de `kubectl logs deploy/self-healing` (quadro do vídeo em 16:25): webhook recebido com status `firing`, pedido de `rollout restart` do `auth-service` aceito pela API do Kubernetes (status 200) e, um minuto depois, o webhook de resolução. A troca efetiva do pod aparece no vídeo em 15:15.

Trecho do mesmo log, em texto, referente ao incidente da gravação:

```
{"time": "2026-11-13T01:53:20.022843+00:00", "level": "INFO", "message": "webhook recebido", "status":
"firing", "alerts": 1}
{"time": "2026-11-13T01:53:20.023421+00:00", "level": "WARNING", "message": "executando rollout restart",
"alertname": "AuthService5xxErrorRate", "namespace": "togglemaster", "deployment": "auth-service",
"restarted_at": "2026-11-13T01:53:20Z"}
{"time": "2026-11-13T01:53:20.045350+00:00", "level": "WARNING", "message": "rollout restart concluído",
"status": 200, "alertname": "AuthService5xxErrorRate", "namespace": "togglemaster", "deployment": "auth-
service", "restarted_at": "2026-11-13T01:53:20Z"}
{"time": "2026-11-13T01:54:20.023003+00:00", "level": "INFO", "message": "webhook recebido", "status":
"resolved", "alerts": 1}
{"time": "2026-11-13T01:54:20.023509+00:00", "level": "INFO", "message": "ação ignorada", "alertname":
"AuthService5xxErrorRate", "deployment": "auth-service", "reason": "alerta não está em firing"}
```
